

INTELLITRACE

AI-POWERED FUND FLOW INTELLIGENCE & BANKING FRAUD DETECTION

FULL IMPLEMENTATION PLAN

30+

MVP Features

50+

AI/ML Models

7

Core Modules

90

Implementation Days Phase
1

Document Version

1.0 | Confidential

Prepared For

Hackathon Round 2 — IntelliTrace Team

Date

May 21, 2026

Classification

Top Secret — Internal Use Only

00

Table of Contents

Complete Navigation Guide

01 Executive Summary

- › Vision & Mission Statement
- › Problem Statement
- › Solution Overview
- › Competitive Differentiation
- › Key Success Metrics

02 System Architecture Overview

- › High-Level Architecture Diagram
- › Component Interaction Map
- › Data Flow Architecture
- › Deployment Topology
- › Technology Stack Summary

03 Data Ingestion Layer

- › Multi-Source Ingestion Strategy
- › Banking Channel Connectors
- › CDC & Event Streaming
- › Data Normalization Schema
- › Error Handling & Retry

04 Identity Resolution Engine

- › Entity Resolution Problem
- › Probabilistic Linking Algorithms
- › Graph-Based Identity Fusion
- › KYC Integration
- › PAN/Aadhaar Linkage

05 Transaction Graph Engine

- › Graph Data Model
- › Node & Edge Schema
- › Graph Query Patterns
- › Temporal Graph Design

- › Neo4j Implementation

06 Real-Time Stream Processing

- › Kafka Architecture
- › Flink Stream Jobs
- › CEP Rule Engine
- › State Management
- › Latency Targets

07 AML Detection Modules

- › Round-Trip Detection
- › Smurfing Detection
- › Rapid Layering
- › Dormant Account Activation
- › Profile Mismatch Engine
- › Mule Account Detection
- › Velocity Anomaly Engine

08 AI & Machine Learning Stack

- › Model Architecture Overview
- › XGBoost Risk Scoring
- › Graph Neural Networks (GraphSAGE)
- › Isolation Forest Anomaly Detection
- › Transformer Sequence Models
- › SHAP Explainability Layer
- › Siamese Network Entity Matching
- › Model Training Pipeline

09 Visualization & Investigator UI

- › Graph Visualization (Cytoscape.js)
- › Sankey Flow Diagrams
- › Timeline Intelligence View
- › Geo-Movement Maps
- › Alert Dashboard
- › Case Management

10 FIU Evidence Package Generation

- › Auto-Generated Investigation Reports
- › STR/SAR Generation
- › Regulatory XML/PDF Templates
- › Evidence Chain of Custody

- › FIU-IND Compliance

11 Compliance & Audit Framework

- › Immutable Audit Logging
- › RBI Regulatory Alignment
- › FATF Compliance Checklist
- › Data Privacy & DPDP Act
- › Explainability Requirements

12 Infrastructure & DevOps

- › On-Premise Deployment
- › Cloud Deployment (AWS/Azure)
- › Kubernetes Orchestration
- › CI/CD Pipeline
- › Monitoring & Observability

13 Implementation Roadmap

- › Phase 1 — Foundation (0–90 Days)
- › Phase 2 — Intelligence (91–180 Days)
- › Phase 3 — Enterprise (181–360 Days)
- › Sprint Planning
- › Team Structure & Roles

14 Risk & Mitigation

- › Technical Risks
- › Regulatory Risks
- › Data Privacy Risks
- › Operational Risks
- › Mitigation Strategies

15 Appendix

- › Glossary of Terms
 - › API Reference Summary
 - › Model Performance Benchmarks
 - › Reference Architecture Diagrams
 - › Bibliography
-

01

Executive Summary

Vision, Problem Statement, Solution & Competitive Differentiation

Vision & Mission

IntelliTrace is an AI-powered, graph-native financial intelligence platform designed to provide end-to-end fund flow visibility across the entire banking ecosystem. Our mission is to make financial crime invisible — to detect, trace, and dismantle money laundering networks before they exploit the financial system. We aim to be India's most advanced AML intelligence engine, purpose-built for the complexity of modern multi-channel, multi-institution payment networks.

Core Mission Statement

To deliver real-time, graph-powered, AI-driven anti-money laundering intelligence that maps every rupee across every channel, detects every laundering pattern, and generates FIU-ready evidence packages — making financial crime detection 10x faster and 90% more accurate than legacy rule-based systems.

Problem Statement

The Indian banking system processes over 13 billion UPI transactions annually, alongside trillions of rupees in NEFT, RTGS, IMPS, SWIFT, and card transactions. Yet, despite this scale, financial crime detection remains fundamentally broken:

- Fragmented Detection:** Banks operate in silos. No unified view of cross-institution money movement exists, allowing sophisticated laundering operations to exploit gaps between financial institutions.
- Static Rule Engines:** Legacy rule-based systems generate 95%+ false positives, creating investigator fatigue and causing genuine threats to be buried under noise.
- Reactive Posture:** Most banks detect fraud hours or days after it occurs. Criminals move money in minutes — often completing entire layering cycles before any alert fires.
- No Graph Intelligence:** Traditional SQL-based systems cannot model complex networks of mule accounts, shell companies, and beneficiary chains that modern laundering operations use.
- Poor Investigator Tooling:** Financial investigators spend 80% of their time gathering evidence manually. There is no automated FIU evidence package generation capability in legacy systems.

X Regulatory Exposure: FIU-IND reports are filed manually, error-prone, and often delayed, exposing banks to regulatory penalties and reputational risk.

Solution Overview

IntelliTrace solves these problems through a seven-layer architecture that combines streaming data ingestion, graph-native data modeling, hybrid AI/ML intelligence, and automated regulatory reporting into a single unified platform:

01	Unified Ingestion	Normalize all banking channels into one event stream
02	Identity Resolution	Link 17+ attributes to unify entities across institutions
03	Transaction Graph	Model money movement as evolving temporal graphs
04	Stream Intelligence	Detect anomalies in milliseconds with Flink CEP
05	ML Risk Engine	XGBoost + GNN + Isolation Forest hybrid scoring
06	Investigator UI	Cytoscape.js graph explorer with Sankey flow & timeline
07	Evidence Package	One-click FIU/STR/SAR generation with full audit trail

Competitive Differentiation

Capability	Legacy AML	Competing Startups	IntelliTrace
Graph Intelligence	X None	Partial	✓ Full Temporal Graph
Real-Time Detection	Batch (hours)	Near-real-time	✓ Milliseconds
Cross-Channel Linking	Single channel	2-3 channels	✓ 8+ channels unified
Entity Resolution	Rule-based only	Basic ML	✓ Probabilistic Graph ML
FIU Auto-Evidence	Manual	Semi-manual	✓ One-click generation
Explainable AI	X Black box	Partial SHAP	✓ Full SHAP + Narrative
Investigator Feedback	X None	Limited	✓ Continuous retraining
False Positive Rate	>95%	60–80%	Target <20%

Key Success Metrics

<20% False Positive Rate	<50ms Alert Latency	99.9% System Uptime	1M+ TPS Capacity
---	--	---	--

02

System Architecture Overview

High-Level Architecture, Data Flow & Technology Stack

High-Level Architecture

IntelliTrace follows a layered, event-driven microservices architecture. Each layer is independently scalable, fault-tolerant, and loosely coupled via Kafka event streams. The architecture is designed for on-premise deployment at banks while supporting hybrid cloud topologies for analytics workloads.

Layer 1 — Data Sources	UPI • NEFT • RTGS • IMPS • SWIFT • Cards • Wallets • Crypto • KYC • Device Telemetry
Layer 2 — Ingestion Bus	Apache Kafka • Pulsar • Debezium CDC • REST/gRPC APIs • Schema Registry
Layer 3 — Processing	Apache Flink Stream Jobs • Flink CEP • RocksDB State • Spark Batch Analytics
Layer 4 — Intelligence	Neo4j Graph DB • XGBoost ML • GraphSAGE GNN • Isolation Forest • SHAP Explainability
Layer 5 — Storage	PostgreSQL (OLTP) • Elasticsearch (Search) • ClickHouse (Analytics) • S3 (Cold Storage)
Layer 6 — Presentation	React Dashboard • Cytoscape.js Graph • D3.js Sankey • Mapbox Geo Views
Layer 7 — Compliance	FIU Evidence Engine • STR/SAR Generator • Audit Logger • RBI Reporting

Data Flow Architecture

01	Transaction Initiation	Customer initiates payment via any channel (UPI, card, NEFT, etc.). Event is captured at source system via CDC or API hook.
02	Event Normalization	Raw transaction is normalized to IntelliTrace Universal Transaction Schema (IUTS) by channel-specific adapters.
03	Kafka Ingestion	Normalized event is published to Kafka topic. Schema validation via Confluent Schema Registry ensures data quality.

04	Identity Resolution	Flink job resolves entity identities in real-time: links customer, account, device, IP to existing graph nodes.
05	Graph Update	Transaction creates/updates nodes and edges in Neo4j. Temporal properties are attached to all relationship records.
06	CEP Rule Evaluation	Flink CEP engine evaluates 200+ rules in parallel. Violations trigger risk events to scoring pipeline.
07	ML Scoring	XGBoost + GNN pipeline generates composite risk score (0–100) with feature importance vector.
08	Alert Generation	Scores above threshold generate alerts with SHAP explanation, linked graph subgraph, and entity timeline.
09	Investigator Routing	Alerts are routed to investigators via case management. High-severity alerts trigger automated evidence packaging.
10	FIU Reporting	Confirmed suspicious activity automatically populates STR/SAR template and submits to FIU-IND portal.

Complete Technology Stack

Component	Primary Technology	Alternatives Considered	Rationale
Event Streaming	Apache Kafka 3.x	Pulsar, RabbitMQ	Ecosystem maturity, banking adoption
Stream Processing	Apache Flink 1.18	Spark Streaming	True stream, stateful CEP support
Graph Database	Neo4j Enterprise	TigerGraph, JanusGraph	Cypher query, APOC library
OLTP Database	PostgreSQL 16	MySQL, CockroachDB	ACID, JSON support, proven scale
Search & Analytics	Elasticsearch 8	OpenSearch, Solr	Full-text, aggregation, APM
Columnar Analytics	ClickHouse	Redshift, BigQuery	Sub-second analytics at petabyte scale
ML Framework	XGBoost + PyTorch	TensorFlow, scikit-learn	Speed + deep learning flexibility
Graph ML	PyTorch Geometric	DGL, Spektral	GraphSAGE, GAT implementation
Explainability	SHAP + LIME	ANCHOR, InterpretML	Industry standard for banking AI

Frontend	React 18 + TypeScript	Vue, Angular	Component ecosystem, performance
Graph Viz	Cytoscape.js	Sigma.js, D3-Force	Banking-grade graph exploration
Container Orch.	Kubernetes + Helm	Docker Swarm	Auto-scaling, rolling deploys
Service Mesh	Istio	Linkerd, Consul	mTLS, circuit breaking, observability
Monitoring	Prometheus + Grafana	Datadog, New Relic	Open source, on-premise friendly

03

Data Ingestion Layer

Multi-Source Banking Data Ingestion, Normalization & Streaming

Multi-Source Ingestion Strategy

The ingestion layer is the foundation of IntelliTrace. It must handle heterogeneous data formats, varying latencies, and strict data quality requirements across 8+ banking channels. The strategy employs a universal adapter pattern where channel-specific connectors translate raw events into the IntelliTrace Universal Transaction Schema (IUTS) before publishing to Kafka.

Source	Protocol	Format	Latency Target	Volume Est.
UPI (NPCI)	REST API + Webhook	JSON	<10ms	5000 TPS
NEFT/RTGS	ISO 20022 XML	XML/FIN	<100ms	500 TPS
IMPS	CBS API	JSON/XML	<10ms	2000 TPS
SWIFT	SWIFT FIN/MX	MT/MX messages	<500ms	100 TPS
Debit/Credit Cards	ISO 8583	Binary/JSON	<50ms	3000 TPS
Mobile Wallets	REST API	JSON	<10ms	1000 TPS
ATM Transactions	NCR/Diebold API	ISO 8583	<100ms	200 TPS
Internet Banking	CBS Event Stream	JSON	<50ms	500 TPS
KYC Systems	CKYC API	JSON/XML	Batch+RT	100 RPS
Device Telemetry	SDK/Beacon	JSON	<10ms	5000 EPS

Universal Transaction Schema (IUTS)

Every ingested transaction is normalized to IUTS before entering the Kafka pipeline. This schema ensures all downstream systems operate on consistent, enriched data.

Field	Type	Description	Source
txn_id	UUID	Global unique transaction ID	System-generated
txn_timestamp	ISO8601	Transaction time (UTC)	Source system

channel	ENUM	UPI/NEFT/RTGS/IMPS/S WIFT/CARD/ATM	Adapter
amount_inr	DECIMAL(18,2)	Normalized amount in INR	FX converted
debit_account_id	STRING	Source account identifier	Source
credit_account_id	STRING	Destination account identifier	Source
debit_entity_id	STRING	Resolved entity ID (sender)	Identity engine
credit_entity_id	STRING	Resolved entity ID (receiver)	Identity engine
debit_bank_ifsc	STRING	Source bank IFSC code	Source
credit_bank_ifsc	STRING	Destination bank IFSC code	Source
device_fingerprint	STRING	Device hash (if available)	Telemetry
ip_address	STRING	Initiating IP address	Source
geo_lat/geo_lon	FLOAT	GPS coordinates (if available)	Device SDK
risk_prelim_score	FLOAT	Pre-computed rule score	Rules engine
metadata_json	JSONB	Channel-specific raw metadata	Adapter

CDC & Change Data Capture

For banks using core banking systems (CBS) like Finacle or BaNCS, Change Data Capture (CDC) via Debezium is used to capture every database write as a Kafka event without modifying the CBS. This is a zero-impact, low-latency integration pattern.

CDC Integration Pattern

Debezium monitors PostgreSQL/Oracle WAL logs → transforms row changes into Kafka events → IntelliTrace adapter normalizes to IUTS → published to transactions.raw Kafka topic. Average latency: <5ms from DB write to Kafka publish. No modification to CBS required.

Error Handling & Data Quality

- Schema validation via Confluent Schema Registry (Avro/Protobuf enforcement)
- Mandatory field checks: txn_id, timestamp, amount, debit/credit accounts
- Duplicate detection using Redis bloom filter (99.99% accuracy)

- Out-of-order event handling via watermarking (5-second tolerance window)
- Dead letter queue (DLQ) for failed events with automatic retry (3 attempts)
- Data lineage tracking: every record stamped with ingestion metadata
- PII masking at ingestion boundary (account numbers hashed for analytics tier)

04

Identity Resolution Engine

Probabilistic Entity Linking Across Banking Ecosystem

The Identity Resolution Challenge

Identity resolution is the hardest and most strategically valuable problem in AML. A sophisticated money laundering operation may use 17 bank accounts across 5 banks, 3 mobile numbers, 6 devices, 4 email addresses, shell companies, and relatives' accounts — all to obscure that a single criminal actor controls the entire network. IntelliTrace's Entity Resolution Engine (ERE) links these disparate identifiers probabilistically to create a unified entity graph.

Key Insight

The entity resolution engine is IntelliTrace's primary competitive moat. Whoever resolves entities most accurately wins — because all downstream ML, graph analytics, and detection logic depends on accurate entity linkage. Our target: link entities with >92% precision at <8% false positive rate.

Identity Attributes Used for Linking

Attribute	Weight	Type	Source
PAN Number	0.95	Hard identifier	CKYC, CBS
Aadhaar (masked)	0.90	Hard identifier	UIDAI API
Mobile Number	0.85	Soft identifier	CBS, KYC
Email Address	0.75	Soft identifier	CBS, KYC
Device Fingerprint	0.80	Behavioral	SDK telemetry
IP Address (subnet)	0.55	Behavioral	Transaction logs
Bank Account Number	0.95	Hard identifier	CBS
IFSC + Account pattern	0.70	Structural	Transaction data
Beneficiary overlap	0.85	Network-based	Graph analysis
Address similarity	0.65	Fuzzy match	KYC records

Name fuzzy match	0.60	Soft identifier	KYC records
VPA (UPI handle)	0.88	Soft identifier	NPCI data
Transaction timing pattern	0.70	Behavioral	Stream analytics
Merchant pattern	0.65	Behavioral	Transaction data
Location clustering	0.60	Behavioral	Geo telemetry

Probabilistic Linking Algorithm

IntelliTrace uses a two-stage linking pipeline: (1) candidate generation via blocking on high-confidence identifiers, followed by (2) pairwise similarity scoring using a trained Siamese neural network.

Stage 1: Blocking

- Hash-based blocking on PAN, Aadhaar suffix, mobile number
- Inverted index on device fingerprint, IP subnet
- MinHash LSH for address and name approximate matching
- Reduces $O(n^2)$ comparisons to $O(n \log n)$

Stage 2: Pairwise Scoring

- Siamese network with 128-dim entity embeddings
- Feature vector: [hard_id_match, soft_id_similarity, behavioral_cosine, graph_overlap]
- Threshold: score $> 0.85 \rightarrow$ definite link, $0.65\text{--}0.85 \rightarrow$ probable link
- Sub-threshold matches flagged for human review

Stage 3: Graph Merge

- Confirmed links create LINKED_TO edges in Neo4j
- Transitive closure: if $A \rightarrow B$ and $B \rightarrow C$, then A-C cluster formed
- Cluster confidence score maintained as edge property
- Periodic re-evaluation as new evidence arrives

05

Transaction Graph Engine

Graph Data Model, Temporal Graph Design & Neo4j Implementation

Why Graph Over Relational?

Traditional relational databases model transactions as rows in tables. This makes it computationally expensive (multi-join queries, poor performance at depth) to answer questions like: 'Show all accounts reachable within 5 hops from Account X' or 'Find all cycles involving this entity.' Graph databases answer these in milliseconds because relationships are first-class citizens, stored as direct pointers, not computed joins.

Query Type	PostgreSQL	Neo4j (IntelliTrace)
2-hop account traversal	~8 seconds (3 JOINS)	<50ms
Cycle detection (fraud ring)	Requires custom SQL	<200ms
5-hop path analysis	Timeout / OOM	<500ms
Community detection	Not supported natively	<2 seconds
Shortest path between entities	Dijkstra via extension	<100ms
Subgraph pattern matching	Complex recursive CTE	<300ms

Graph Node Schema

:Customer	customer_id (PK), entity_cluster_id, kyc_status, risk_score, customer_since, segment
:Account	account_id (PK), ifsc_code, account_type, open_date, current_balance, dormancy_days
:Transaction	txn_id (PK), amount_inr, txn_timestamp, channel, status, risk_score
:Device	device_id (PK), device_type, os, imei, first_seen, last_seen
:IPAddress	ip_address (PK), isp, country, is_vpn, is_tor, risk_label
:Merchant	merchant_id (PK), merchant_category, registered_name, risk_tier
:Company	company_id (PK), cin_number, incorporation_date, director_ids, ubo_ids

:Wallet	wallet_id (PK), provider, kyc_level, linked_account_ids
---------	---

Graph Relationship Schema

Relationship	From Node	To Node	Key Properties
OWNS	Customer	Account	since_date, joint_holder
TRANSFERRED_TO	Account	Account	amount, txn_id, timestamp, channel
INITIATED_FROM	Transaction	Device	session_id, timestamp
LOGGED_IN_USING	Customer	IPAddress	last_seen, frequency
LINKED_TO	Customer	Customer	link_confidence, link_type
BENEFICIARY_OF	Account	Customer	transfer_count, total_amount
OPERATES_AT	Customer	Merchant	frequency, avg_amount
CONTROLS	Customer	Company	role, ownership_pct
USES_WALLET	Customer	Wallet	linked_since, txn_count
CONNECTED_VIA	Account	IPAddress	access_count, last_seen

Temporal Graph Design

Most AML graph implementations are static — they capture the current state of the world. IntelliTrace implements a temporal graph where every relationship carries time-series properties. This enables time-windowed graph queries critical for detecting rapid layering (events compressed in minutes) and historical pattern analysis.

Temporal Edge Example

TRANSFERRED_TO relationship properties: {amount: 50000, timestamp: '2024-01-15T09:03:42Z', valid_from: '2024-01-15T09:03:42Z', valid_to: null, channel: 'UPI', sequence_in_chain: 3, hop_count_from_origin: 3, time_since_origin: 142}

06

Real-Time Stream Processing

Kafka Architecture, Flink CEP, State Management & Latency Targets

Kafka Architecture

Apache Kafka serves as the central nervous system of IntelliTrace. Every transaction, alert, entity update, and model score flows through Kafka topics. This decouples producers from consumers, provides replay capability, and enables multiple processing pipelines to consume the same events independently.

Kafka Topic	Partitions	Retention	Consumers	Purpose
intellitrace.txns.raw	96	7 days	Adapter, QA	Raw normalized transactions
intellitrace.txns.enriched	96	30 days	Flink, Neo4j	Enriched with entity IDs
intellitrace.alerts.raw	32	90 days	Case Mgmt, ML	Raw alert events
intellitrace.alerts.scored	32	90 days	Investigator UI	Scored & ranked alerts
intellitrace.entity.updates	48	30 days	Neo4j, Redis	Entity graph changes
intellitrace.risk.scores	48	30 days	Dashboard, FIU	Real-time risk scores
intellitrace.audit.events	24	365 days	Audit DB	Immutable audit trail
intellitrace.fiu.reports	8	365 days	FIU Engine	Report generation queue

Flink Stream Processing Jobs

TransactionEnricher

Enriches raw transactions with entity IDs, geo-lookup, device history. Latency: <5ms.

GraphUpdater	Updates Neo4j graph with new transactions. Batches updates for efficiency. Latency: <50ms.
RuleEvaluator	Evaluates 200+ stateless rules against each transaction. Latency: <10ms per rule.
CEP Detector	Flink Complex Event Processing for multi-event patterns (smurfing, layering). Window: 5min/1hr.
VelocityCalculator	Maintains rolling windows (1min, 15min, 1hr, 24hr) of account activity metrics.
DormancyMonitor	Tracks account inactivity periods. Fires alerts on activation after >30/90/180 day dormancy.
ProfileAnalyzer	Compares current transaction patterns against customer behavioral baseline. Z-score calculation.
MLScorer	Calls XGBoost scoring service with pre-computed feature vector. Latency: <20ms.
AlertAggregator	Deduplicates and correlates alerts from multiple detectors. Builds composite alert.
AuditLogger	Writes immutable audit records for every processing step. Cryptographically signed.

CEP Pattern Examples

Smurfing Pattern

PATTERN: Sequence of >10 credits within 2 hours, each < ₹50,000, to same account OR same cluster of accounts, sum > ₹5 lakhs. Window: 2 hours sliding. State: per-account credit accumulator.

Rapid Layering Pattern

PATTERN: Account receives >₹1 lakh AND disperses >80% to >5 accounts within 30 minutes. Multi-hop: same pattern repeating within 1 hour. Window: 30-minute tumbling. State: per-account debit/credit tracker.

Round Trip Pattern

PATTERN: Graph cycle detected where funds return to origin account within 24 hours through ≥ 3 intermediaries. Detection: Neo4j cycle detection triggered after graph update.

Dormant Activation Pattern

PATTERN: Account with 0 transactions in >180 days receives >₹5 lakhs AND initiates outward transfer within 2 hours. Simple event sequence with dormancy state maintained in RocksDB.

07

AML Detection Modules

Round-Trip, Smurfing, Layering, Dormancy, Profile Mismatch & Mule Detection

Round-Trip Detection

RISK: CRITICAL

Detects money that exits an account and returns through intermediaries, indicating circular movement used to create paper trails of legitimate transactions.

Algorithm: Tarjan's Strongly Connected Components (SCC) on temporal transaction graph. Identifies cycles where same entity appears as both source and destination within a configurable time window (default: 72 hours).

Detection Features: Cycle length (number of hops) • Time compression ratio (total cycle time / expected legitimate time) • Amount preservation ratio (exit amount / entry amount) • Number of distinct intermediary entities • Intermediary account age and risk scores

Threshold Rule: Alert if: $\text{cycle_length} \geq 3$, $\text{time_window} \leq 72\text{h}$, $\text{amount_preserved} \geq 85\%$

Smurfing / Structuring Detection

RISK: HIGH

Detects deliberate fragmentation of large amounts into smaller transactions to avoid reporting thresholds (₹10 lakh for banks, ₹50,000 for wallets).

Algorithm: Isolation Forest on transaction fragmentation features, combined with Flink CEP aggregation window. XGBoost classifier trained on confirmed structuring cases.

Detection Features: Transaction fragmentation score (std dev of amounts) • Threshold proximity ratio (amount / reporting_threshold) • Burst interval (transactions per hour) • Beneficiary entropy (diversity of recipients) • Cross-channel fragmentation (same amount across UPI + NEFT + IMPS)

Threshold Rule: Alert if: $\text{fragment_count} \geq 10$, $\text{sum} > ₹5\text{L}$, $\text{avg_amount} < ₹50\text{K}$, $\text{interval} < 120\text{min}$

Rapid Layering Detection

RISK: CRITICAL

Detects rapid multi-hop movement of funds through multiple accounts designed to obscure the origin and create complex audit trails.

Algorithm: Temporal Graph Neural Network (GraphSAGE with time-aware aggregation). Graph features extracted from 3-hop neighborhood with temporal weighting.

Detection Features: Hop count within time window • Temporal compression score • Fan-out ratio at each layer • Amount decay ratio per hop • Network centrality of intermediate accounts

Threshold Rule: Alert if: $\text{hops} \geq 4$, $\text{time_window} \leq 30\text{min}$, $\text{fan_out} \geq 5$ at any layer

Dormant Account Activation

RISK: HIGH

Classic mule account indicator. Accounts that are inactive for extended periods, then suddenly activated with large transfers and rapid dispersal.

Algorithm: Rule-based primary detection with XGBoost secondary scoring on activation context features.

Detection Features: Days since last transaction • Activation amount vs. historical average • Time to first outward transfer post-activation • Number of beneficiaries in first 24h post-activation • Whether beneficiaries are newly created accounts

Threshold Rule: Alert if: dormancy > 90 days AND activation_amount > 10x historical_avg AND outward_transfer < 2h

Profile Mismatch Detection

RISK: HIGH

Detects when actual transaction behavior contradicts the declared customer profile (income, occupation, transaction purpose).

Algorithm: Behavioral baseline model (XGBoost with SHAP) trained per customer segment. Z-score anomaly on key behavioral features.

Detection Features: Monthly transaction volume vs. declared income • Transaction geographies vs. declared residence • Counterparty types vs. declared occupation • Cash-equivalent withdrawal ratio • International transfer frequency vs. declared business

Threshold Rule: Alert if: any feature Z-score > 3.5 AND composite anomaly score > 0.80

Mule Account Network Detection

RISK: CRITICAL

Identifies coordinated networks of accounts used to receive and disperse laundered funds, often operated by recruited 'mules'.

Algorithm: Community detection (Louvain algorithm) on transaction graph, followed by GNN classification of detected communities.

Detection Features: Account turnover ratio (outflows / inflows) • Balance retention time (how quickly funds are moved out) • Number of unique counterparties • Community centrality score • Shared device/IP with other cluster members

Threshold Rule: Alert if: turnover_ratio > 0.95 AND balance_retention < 30min AND unique_counterparties > 20

08

AI & Machine Learning Stack

Hybrid ML Architecture, Model Selection, Training Pipeline & Explainability

ML Architecture Philosophy

IntelliTrace's ML strategy is deliberately hybrid. Pure deep learning fails in banking AML because regulators require explainability, training data is heavily imbalanced (<0.1% fraud), and models must be auditable. Our architecture combines fast, interpretable tree models for per-transaction scoring with graph neural networks for network-level detection.

XGBoost — Core Risk Scoring Engine

Role	Primary per-transaction risk scorer
Why Selected	Best-in-class performance on tabular banking data. Sub-millisecond inference. Native SHAP explainability. Handles class imbalance via scale_pos_weight.
Input Features	200+ engineered features including: velocity metrics, behavioral Z-scores, account age, channel diversity, beneficiary risk, geo-anomaly signals, KYC mismatch indicators.
Training Strategy	Supervised on labeled fraud/non-fraud transactions. SMOTE oversampling for minority class. 5-fold stratified cross-validation. Target AUC > 0.96 on holdout.
Inference Latency	<5ms per transaction including feature computation.

GraphSAGE — Network Laundering Detector

Role	Graph-level laundering pattern detection
Why Selected	Captures neighborhood structure of accounts in the transaction graph. Detects mule networks, layering clusters, and hidden beneficial ownership structures that tabular models miss entirely.
Input Features	Node features: account metrics, velocity, profile. Edge features: amount, time, channel. 3-hop neighborhood aggregation with mean pooling.
Training Strategy	Semi-supervised: labeled confirmed fraud subgraphs + self-supervised link prediction on unlabeled graph. Re-trained weekly with investigator feedback.
Inference Latency	<50ms for 3-hop neighborhood (cached embeddings for known nodes).

Isolation Forest — Anomaly Detection

Role	Unsupervised anomaly scoring for novel patterns
Why Selected	Effective on high-dimensional behavioral data with no labeled examples. Detects novel laundering techniques not yet in training data. Complements supervised XGBoost with unsupervised signal.
Input Features	Behavioral feature vectors: transaction velocity, amount distributions, temporal patterns, channel mix, counterparty diversity.
Training Strategy	Trained on 'clean' customer baseline behavior. Contamination parameter tuned to expected fraud rate (~0.5%). Monthly re-training.
Inference Latency	<2ms per transaction.

Transformer — Sequence Pattern Detector

Role	Temporal sequence anomaly detection
Why Selected	Self-attention mechanism captures long-range dependencies in transaction sequences. Detects complex multi-step laundering sequences like: cash_in → split → crypto → offshore.
Input Features	Transaction sequence encoded as tokens: [channel, amount_bucket, counterparty_type, time_delta]. Sequence length: 50 transactions.
Training Strategy	Self-supervised pre-training via masked transaction prediction. Fine-tuned on labeled laundering sequences.
Inference Latency	<30ms for 50-transaction window.

SHAP — Explainability Engine

Role	Model explanation for investigators and regulators
Why Selected	Banks require every ML alert to be explainable. SHAP provides feature-level attribution that tells investigators exactly WHY a transaction scored high.
Input Features	TreeExplainer for XGBoost (fast, exact). DeepExplainer for neural models (approximate). SHAP values aggregated into human-readable risk factors.
Training Strategy	No training — applied post-hoc to any ML model.
Inference Latency	<10ms for XGBoost SHAP. Used on every alert generated.

09

Visualization & Investigator UI

Graph Explorer, Sankey Flows, Timeline Views & Alert Dashboard

Investigator Workflow Design Principles

The investigator UI is purpose-built for financial crime analysts. Every design decision optimizes for: (1) speed of case resolution — target from alert to FIU submission in <2 hours, (2) evidence clarity — every visualization must be presentable in regulatory hearings, (3) zero training required — investigators should be productive within 30 minutes.

Graph Explorer (Cytoscape.js)

- Interactive node-link diagram of account transaction networks
 - 1-click expand: show all connected entities up to N hops
 - Path tracing: highlight shortest/most suspicious path between two entities
 - Community overlay: color-code detected mule clusters
 - Temporal replay: animate transaction flow over time slider
 - Export: PNG/SVG for evidence packages
 - Layout algorithms: force-directed, circular, hierarchical
-

Sankey Fund Flow Visualizer

- Stacked flow diagram showing money movement volume across channels
 - Source → Intermediary → Destination with amount proportional bandwidth
 - Split visualization for smurfing pattern evidence
 - Time-windowed filtering (1h, 24h, 7d, custom)
 - Entity labels with risk score color coding
 - D3.js + React implementation, export to SVG
-

Timeline Intelligence View

- Vertical timeline of all transactions for selected entity/case
- Event markers: anomalies, threshold crossings, channel switches
- Overlaid risk score line chart with threshold annotations
- Multi-entity comparison: side-by-side timelines

- Zoom to seconds granularity for rapid layering analysis
-

Alert Dashboard

- Real-time alert feed with severity, type, entity, amount
 - Prioritization: sorted by composite risk score × financial exposure
 - One-click to case: open full investigation from alert
 - Team queue management: assign, escalate, resolve
 - KPI widgets: daily alert count, resolution rate, FP rate, FIU submissions
-

Geo-Intelligence Map

- Mapbox/Leaflet integration showing transaction geography
 - Impossible travel detection overlay
 - Branch heat map: suspicious activity by location
 - ATM cash-out geographic clustering
 - International transfer routing visualization
-

Case Management System

- Full lifecycle management: Alert → Investigation → Resolution → FIU Filing
 - Evidence attachment: graph snapshots, timeline exports, AI narrative
 - Collaboration: notes, @mentions, escalation trails
 - SLA tracking: time-to-resolution dashboards for compliance
 - Audit trail: every investigator action immutably logged
-

10

FIU Evidence Package Generation

Auto-Generated STR/SAR Reports, Regulatory Compliance & Chain of Custody

Why Evidence Generation Is A Strategic Differentiator

Most AML systems stop at generating alerts. The investigator then manually spends 4–8 hours compiling evidence from multiple systems to file a Suspicious Transaction Report (STR). IntelliTrace automates this entirely — from confirmed alert to FIU-ready evidence package in under 5 minutes. This is our strongest enterprise sales differentiator.

Business Impact of Evidence Automation

If a bank investigates 500 cases/month and each takes 4 hours manually, that is 2,000 investigator-hours/month. IntelliTrace reduces this to 30 minutes/case = 250 hours. ROI: 1,750 investigator-hours saved per month (~₹35 lakh monthly savings for mid-size bank).

Auto-Generated Investigation Package Components

01	Executive Summary AI-generated 300-word narrative summarizing the suspicious pattern, involved entities, financial exposure, and confidence level. Generated by fine-tuned language model with banking-specific vocabulary.
02	Transaction Timeline Chronological table of all transactions in the case with amounts, channels, entities, timestamps, and anomaly flags. Formatted for FIU STR Section 3.
03	Entity Profile Sheet For each involved entity: KYC data, account history summary, risk score history, declared vs. actual profile comparison, linked entities.
04	Graph Visualization High-resolution PDF export of Cytoscape.js graph showing complete fund flow network with entity labels and amount annotations.
05	Sankey Flow Diagram Visual fund flow from source to final destination with all intermediaries, amounts, and timestamps.

06	ML Explainability Report SHAP feature importance breakdown explaining why each entity was flagged. 'Alert score 94 because: dormancy violation (contribution: 0.31), rapid dispersal (0.28), mule cluster proximity (0.22)...'
07	Device & IP Intelligence All devices and IPs linked to involved accounts, with geolocation, VPN/TOR flags, and cross-entity device sharing analysis.
08	Regulatory Metadata Bank details, reporter details, date/time of suspicion, date/time of report generation. Pre-filled FIU-IND STR XML.
09	Chain of Custody Log Immutable log: who accessed the case, what actions were taken, when each piece of evidence was generated. Required for legal admissibility.
10	Supporting Annexures Raw transaction logs, KYC document references, model version numbers, threshold values in effect at time of detection.

FIU-IND STR Filing Workflow

- **Alert Confirmed:** Investigator marks alert as suspicious after reviewing evidence
- **Auto-Generation:** System generates complete evidence package in <5 minutes
- **Investigator Review:** 30-minute review and annotation by analyst
- **Supervisor Approval:** One-click approval by compliance supervisor
- **FIU XML Generation:** System generates FIU-IND compliant STR XML
- **Submission:** Auto-submission to FIU-IND portal via API or secure email
- **Acknowledgment:** FIU acknowledgment captured and stored in case record

11

Compliance & Audit Framework

RBI Alignment, FATF Compliance, Data Privacy & Explainability Requirements

Regulatory Landscape

Regulation	Authority	Key Requirements	IntelliTrace Coverage
PMLA 2002 + 2023 Amendments	MoF / FIU-IND	STR filing, record keeping 5yr+, CDD	Full — auto STR, audit logs
RBI AML Master Circular	Reserve Bank of India	Enhanced CDD, wire transfer rules	Full — KYC integration
FATF Recommendations	Financial Action Task Force	Risk-based approach, beneficial ownership	Full — UBO detection
DPDP Act 2023	Ministry of IT	Data minimization, consent, localization	Full — PII masking, consent tracking
IT Act 2000 / CERT-In	CERT-In / MeitY	Cybersecurity incident reporting	Partial — SIEM integration
SEBI AML Guidelines	SEBI	Broker AML obligations	Roadmap — Phase 3

Immutable Audit Logging Architecture

Every action in IntelliTrace — transaction processed, alert generated, investigator login, model score computed, evidence package created, FIU report filed — is written to an immutable audit log. These logs are cryptographically signed using SHA-256 hash chains (similar to a blockchain structure) to ensure tamper-evidence.

- event_id (UUID — globally unique)
- event_timestamp (UTC — millisecond precision)
- event_type (ENUM: TRANSACTION_SCORED, ALERT_GENERATED, CASE_CREATED, etc.)
- actor_id (system component or human user who triggered the event)
- entity_ids (list of affected customers/accounts/transactions)
- action_summary (structured JSON describing what happened)
- previous_hash (SHA-256 hash of previous audit record)
- current_hash (SHA-256 hash of this record — ensures chain integrity)

- model_version (if ML event: model ID, version, feature hash)

Explainability Requirements

RBI and FIU-IND require that any ML-based alert must be explainable to regulators. IntelliTrace meets this via: (1) SHAP values for every XGBoost prediction, (2) Natural language explanation generator, (3) Rule attribution for CEP-triggered alerts, (4) Complete feature vector logging for every scored transaction.

Example Explainability Output

Alert ID: ITR-2024-089432 | Score: 94/100 Primary factors: Dormant account activated after 214 days (+0.31), funds dispersed to 18 accounts within 11 minutes (+0.28), 3 beneficiaries linked to known mule cluster (+0.22), transaction velocity 400x customer baseline (+0.13). Recommended action: File STR within 7 days per PMLA Section 16.

12

Infrastructure & DevOps

Kubernetes Deployment, CI/CD Pipeline, Monitoring & High Availability

Deployment Architecture

IntelliTrace supports three deployment modes: (1) On-premise — for banks with data sovereignty requirements, (2) Private cloud — bank-managed AWS/Azure VPC, (3) Hybrid — processing on-premise, analytics in cloud. Most Indian banks will prefer on-premise or private cloud to comply with RBI data localization guidelines.

Microservice	Replicas (Prod)	CPU Request	Memory Request	Scaling Trigger
transaction-ingester	6	2 cores	4 GB	Kafka lag > 10k
identity-resolver	4	4 cores	8 GB	Queue depth > 1k
graph-updater	4	2 cores	4 GB	Write latency > 100ms
flink-cep-engine	8	4 cores	8 GB	Event lag > 5k
ml-scorer-xgb	8	2 cores	2 GB	Queue depth > 500
ml-scorer-gnn	4	4 cores	16 GB	Queue depth > 200
alert-manager	3	1 core	2 GB	Alert rate > 100/min
evidence-generator	3	2 cores	4 GB	Manual/triggered
api-gateway	4	2 cores	2 GB	RPS > 1000
investigator-ui	3	1 core	1 GB	Concurrent users > 50
fiu-reporter	2	1 core	2 GB	Queue depth > 10

CI/CD Pipeline

Code Commit	GitHub/GitLab push triggers pipeline	→
Static Analysis	SonarQube SAST scan, dependency vulnerability check	→
Unit Tests	pytest suite — target >90% code coverage	→

Integration Tests	Testcontainers spin up Kafka + Neo4j for end-to-end tests	→
ML Model Tests	Champion/challenger comparison on held-out validation set	→
Docker Build	Multi-stage Docker build — slim production images	→
Security Scan	Trivy container scan — fail on CRITICAL CVEs	→
Staging Deploy	Helm deploy to staging cluster with canary traffic	→
Smoke Tests	Automated transaction flow tests in staging	→
Production Deploy	Rolling deploy with instant rollback on error rate spike	→
Monitoring	Auto-alert if P99 latency exceeds threshold post-deploy	→

High Availability & Disaster Recovery

RTO (Recovery Time Objective)	< 15 minutes
RPO (Recovery Point Objective)	< 5 minutes (Kafka replay)
Kafka Replication Factor	3 (minimum 2 in-sync replicas)
Neo4j HA Mode	3-node causal cluster with read replicas
PostgreSQL HA	Patroni cluster with auto-failover
Multi-AZ Deployment	Active-active across 2 availability zones
Backup Frequency	Hourly incremental, daily full snapshot to encrypted S3

13

Implementation Roadmap

Phased Delivery Plan, Sprint Structure & Team Composition

Phase 1 — Foundation

Days 1–90 (3 Months)

Goal: Build the core data pipeline, basic graph, and first detection modules. Demonstrate end-to-end flow: ingest → graph → alert → dashboard.

Key Milestones	Deliverables
• Week 1–2: Environment setup, Kafka cluster, schema design	• Working transaction pipeline (UPI, NEFT, IMPS)
• Week 3–4: UPI + NEFT ingestion adapters, IUTS normalization	• Live Neo4j graph with entity resolution
• Week 5–6: Neo4j graph setup, basic entity-account-transaction nodes	• XGBoost risk scorer with SHAP explanations
• Week 7–8: Flink stream jobs (enricher, graph updater, velocity calculator)	• Basic investigator dashboard with alert feed
• Week 9–10: XGBoost baseline model, 50 detection rules in Flink CEP	• 5 detection modules: smurfing, dormancy, velocity, round-trip, layering
• Week 11–12: Alert dashboard (React), basic graph explorer (Cytoscape.js)	

Team: 2 backend engineers, 1 ML engineer, 1 frontend engineer, 1 DevOps

Phase 2 — Intelligence

Days 91–180 (Months 4–6)

Goal: Layer advanced ML models, complete visualization suite, and build FIU evidence generation capability.

Key Milestones	Deliverables
• Week 13–14: SWIFT + Card + Wallet ingestion, full identity resolution engine	• Full 8-channel ingestion pipeline
• Week 15–16: GraphSAGE GNN implementation on Neo4j graph	• GraphSAGE mule network detector
• Week 17–18: Isolation Forest anomaly detection, profile mismatch engine	• Complete visualization suite (5 views)

• Week 19–20: Sankey visualizer, timeline view, geo intelligence map	• FIU STR auto-generation
• Week 21–22: FIU evidence package generator (PDF, XML, narrative)	• Case management with audit trail
• Week 23–24: Case management system, investigator feedback loop	• Investigator feedback → model retraining loop
Team: 3 backend, 2 ML, 2 frontend, 1 DevOps, 1 Compliance specialist	

Phase 3 — Enterprise

Days 181–360 (Months 7–12)

Goal: Production hardening, enterprise integrations, regulatory certification, and first bank pilot deployment.

Key Milestones	Deliverables
• Month 7: Transformer sequence model, temporal GNN upgrade	• Enterprise-grade performance (1M TPS, <50ms end-to-end)
• Month 8: CKYC API integration, Aadhaar eKYC linkage	• Regulatory certifications (RBI, CERT-In)
• Month 9: Multi-bank data sharing framework (with consent layer)	• Multi-bank deployment capability
• Month 10: Performance optimization — target 1M TPS	• Advanced AI copilot for investigators
• Month 11: RBI regulatory review, CERT-In audit, penetration testing	• Complete FATF compliance documentation
• Month 12: Pilot bank deployment, SLA monitoring, support framework	• First paid pilot with Indian bank
Team: Full team: 5 backend, 3 ML, 3 frontend, 2 DevOps, 2 Compliance, 1 BD	

14

Risk & Mitigation

Technical, Regulatory, Data Privacy & Operational Risk Framework

Technical Risks

Graph Database Scale		HIGH
Risk: Neo4j performance degrades at very large graph sizes	Mitigation: Implement graph partitioning, use Neo4j Enterprise clustering, cache hot subgraphs in Redis	
ML Model Drift		MEDIUM
Risk: AML patterns evolve; models degrade without retraining	Mitigation: Weekly model evaluation on production data, automated champion/challenger pipeline, investigator feedback loop	
Kafka Consumer Lag		HIGH
Risk: Transaction volume spikes cause processing delays	Mitigation: Auto-scaling Flink consumers, backpressure handling, DLQ for overflow, capacity planning for 3x expected load	
Entity Resolution Errors		HIGH
Risk: False links between legitimate customers cause false positives	Mitigation: Conservative threshold tuning, human-in-the-loop for ambiguous links, monthly precision/recall audit	

Regulatory Risks

RBI Approval		HIGH
Risk: System may require RBI sign-off before bank deployment	Mitigation: Engage RBI regulatory sandbox, appoint dedicated compliance officer, maintain detailed regulatory documentation	
DPDP Act Compliance		HIGH

Risk: Customer data processing must comply with India DPDP Act 2023	Mitigation: PII tokenization at ingestion, consent management module, data retention policy enforcement, DPO appointment
FIU Reporting Errors MEDIUM	
Risk: Incorrect STR filings carry penalties	Mitigation: Multi-step human review before filing, legal review of first 50 reports, maintain STR audit trail

Operational Risks

High False Positive Rate CRITICAL	
Risk: Investigator overload leads to alert fatigue and missed real cases	Mitigation: Aggressive FP reduction: investigator feedback retraining, adaptive thresholds per customer segment, alert deduplication
Banking Integration Complexity HIGH	
Risk: Each bank CBS integration is unique and time-consuming	Mitigation: Publish open API specification, provide 5 pre-built CBS connectors, professional services team for integration
Data Quality from Banks MEDIUM	
Risk: Banks may provide incomplete or inconsistent transaction data	Mitigation: Data quality dashboard, mandatory field enforcement, SLA with banks on data quality metrics

15

Appendix

Glossary, API Reference, Benchmarks & Bibliography

Glossary of Key Terms

AML	Anti-Money Laundering — regulatory framework and technology to detect financial crimes
CKYC	Central KYC Registry — India's centralized repository of KYC records
STR	Suspicious Transaction Report — mandatory report filed with FIU-IND
SAR	Suspicious Activity Report — US equivalent of STR
FIU-IND	Financial Intelligence Unit India — body receiving and analyzing financial crime reports
FATF	Financial Action Task Force — global AML standards body
PMLA	Prevention of Money Laundering Act — India's primary AML legislation
CEP	Complex Event Processing — real-time pattern detection over event streams
GraphSAGE	Graph Sample and Aggregate — inductive GNN algorithm for graph ML
SHAP	SHapley Additive exPlanations — ML explainability framework
CDC	Change Data Capture — database streaming technique for real-time data pipelines
IUTS	IntelliTrace Universal Transaction Schema — normalized transaction format
Mule Account	Account used by criminals to receive and pass on laundered funds
Layering	Process of moving money through multiple accounts to obscure its origin
Structuring / Smurfing	Breaking large transactions into smaller ones to avoid reporting
Round Tripping	Money that exits and re-enters an account through intermediaries
SCC	Strongly Connected Component — graph subgraph where all nodes are reachable from each other
TPS	Transactions Per Second — measure of system throughput
RTO/RPO	Recovery Time/Point Objective — disaster recovery metrics
DPDP	Digital Personal Data Protection Act 2023 — India's data privacy law

Model Performance Benchmarks (Targets)

Model	Dataset	AUC-ROC	Precision	Recall	F1	Latency
XGBoost (Transaction)	Synthetic + Augmented	>0.96	>88%	>85%	>86%	<5ms
GraphSAGE (Network)	Graph-labeled subgraphs	>0.91	>82%	>79%	>80%	<50ms
Isolation Forest	Unlabeled behavioral data	N/A	Tuned	>75%	N/A	<2ms
Transformer (Sequence)	Sequence-labeled data	>0.89	>80%	>77%	>78%	<30ms
Composite Ensemble	Production validation set	>0.97	>91%	>88%	>89%	<60ms

Key References & Bibliography

[1] FATF. (2023). Guidance on a Risk-Based Approach for Virtual Assets and Virtual Asset Service Providers.

[2] FIU-IND. (2023). Annual Report on Financial Intelligence and Suspicious Transaction Reports.

[3] Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs (GraphSAGE). NeurIPS.

[4] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD.

[5] Lundberg, S., & Lee, S.I. (2017). A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS.

[6] Akoglu, L., Tong, H., & Koutra, D. (2015). Graph-based Anomaly Detection and Description: A Survey.

[7] Reserve Bank of India. (2023). Master Direction on Know Your Customer (KYC).

[8] Ministry of Finance. (2023). Prevention of Money Laundering (Amendment) Act.

[9] NPCI. (2023). UPI Annual Statistics Report.

[10] Palantir Technologies. (2022). Gotham AML Platform Technical Architecture (Public).

[11] Feedzai. (2023). The Financial Crime Report: Trends in AML and Fraud Detection.

[12] Neo4j Inc. (2023). Graph Data Science Library Documentation.

[13] Apache Software Foundation. (2023). Apache Flink Documentation.